

Министерство образования Республики Беларусь
Учреждение образования «Белорусский государственный университет
информатики и радиоэлектроники»

Факультет компьютерных систем и сетей
Кафедра Информатики
Дисциплина «Избранные главы информатики»

ОТЧЕТ
к лабораторной работе №3
на тему:
**«СТАНДАРТНЫЕ ТИПЫ ДАННЫХ, КОЛЛЕКЦИИ, ФУНКЦИИ,
МОДУЛИ»**
БГУИР 6-05-0612-02

Выполнил студент группы 453503
ВОДНЕВ Кирилл Владимирович

(дата, подпись студента)

Проверил доцент каф. Информатики
ЖВАКИНА Анна Васильевна

(дата, подпись преподавателя)

Минск 2026

Задание 1. В соответствии с заданием своего варианта составить программу для вычисления значения функции с помощью разложения функции в степенной ряд. Задать точность вычислений eps.

Предусмотреть максимальное количество итераций, равное 500.

Вывести количество членов ряда, необходимых для достижения указанной точности вычислений. Результат получить в виде:

x	n	$F(x)$	$Math F(x)$	eps

Здесь x – значение аргумента, $F(x)$ – значение функции, n – количество просуммированных членов ряда, $Math F(x)$ – значение функции, вычисленное с помощью модуля math.

Вар-т	Условие
9.	$\arccos x = \frac{\pi}{2} - \arcsin x = \frac{\pi}{2} - \sum_{n=0}^{\infty} \frac{(2n)!}{4^n (n!)^2 (2n+1)} x^{2n+1} = \frac{\pi}{2} - x - \frac{x^3}{6} - \dots, x \leq 1$

Листинг кода файла task1.py для выполнения задания 1 с использованием декоратора:

```
from tabulate import tabulate
import math
import inputValidator

DESCRIPTION = """Write a program to calculate the value of arccos(x)
using its power series expansion: arccos(x) = π/2 - arcsin(x) = π/2 - Σ [(2n)! / (4^n * (n!)^2 * (2n+1))] * x^(2n+1)
where |x| ≤ 1"""

MAX_ITERATION = 500
TITLE_TABLE = ["x", "n", "F(x)", "Math F(x)", "eps"]

def print_description(): 1 usage
    """Function for describing the task condition"""
    print(DESCRIPTION, end= "\n\n")

def print_table(func): 1 usage
    """
        Decorator function for outputting results in a table

        Args:
        | func: Function to wrap

        Returns:
        | Wrapper function
    """

    def wrapper_in_table(*args, **kwargs):
        result = func(*args, **kwargs)
        print(tabulate( tabular_data: [result], headers=TITLE_TABLE, tablefmt="grid", floatfmt=".10f"))
        return result

    return wrapper_in_table
```

```
def get_validate_inputs(): 1 usage
    """
    Function to receive input from the user

    Returns:
    | Tuple of (eps, x) values
    """
    eps = inputValidator.input_data_with_random( description: "Input value from 0 to 1 for eps: ",
                                                float, min_value: 1e-10, max_value: 0.999, is_generate_random=True)
    x = inputValidator.input_data_with_random( description: "Input argument func, where |x| <= 1: ",
                                                float, -1.0, max_value: 1.0, is_generate_random=True)

    return eps, x
```

```
def factorial(n): 2 usages
    """
    Calculate factorial of n

    Args:
    | n: Integer number

    Returns:
    | Factorial of n
    """
    if n <= 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result
```

```
@print_table 1 usage
def calculate(eps, x):
    """
    Function for calculating arccos(x) using series until a specified accuracy is reached

    Args:
    | eps: Desired accuracy
    | x: Argument value

    Returns:
    | Tuple of (x, iterations, calculated_value, math_value, eps)
    """
    math_fx = math.acos(x) # Reference value using math library

    # arccos(x) = π/2 - arcsin(x)
    # arcsin(x) = Σ(n=0 to ∞) [(2n)! / (4^n * (n!)^2 * (2n+1))] * x^(2n+1)

    arcsin_sum = 0
    n = 0

    while n < MAX_ITERATION:
        # Calculate term: [(2n)! / (4^n * (n!)^2 * (2n+1))] * x^(2n+1)
        numerator = factorial(2 * n)
        denominator = (4 ** n) * (factorial(n) ** 2) * (2 * n + 1)
        term = (numerator / denominator) * (x ** (2 * n + 1))

        arcsin_sum += term

        # Check convergence
        if abs(term) < eps:
            break

        n += 1

    fx = (math.pi / 2) - arcsin_sum # arccos(x) = π/2 - arcsin(x)

    return x, n + 1, fx, math_fx, eps
```

Результат выполнения работы:

```
=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 1
Write a program to calculate the value of arccos(x)
using its power series expansion:  $\arccos(x) = \pi/2 - \arcsin(x) = \pi/2 - \sum [(2n)! / (4^n * (n!)^2 * (2n+1))] * x^{(2n+1)}$ 
where  $|x| \leq 1$ 

Input value from 0 to 1 for eps:
Error: could not convert string to float: '. Please enter a valid value.
Input value from 0 to 1 for eps: d
Error: could not convert string to float: 'd'. Please enter a valid value.
Input value from 0 to 1 for eps: 0
Error: The value have to be greater or = then 1e-10. Please enter a valid value.
Input value from 0 to 1 for eps: 0.0000001
Input argument func, where  $|x| \leq 1$ : 0.5
+-----+-----+-----+-----+-----+
|          x |  n |          F(x) |  Math F(x) |          eps |
+-----+-----+-----+-----+-----+
| 0.5000000000 |  9 | 1.0471975749 | 1.0471975512 | 0.0000001000 |
+-----+-----+-----+-----+-----+
```

Задание 2. В соответствии с заданием своего варианта составить программу для нахождения суммы последовательности чисел.

Вар-т	Условие
9.	Организовать цикл, который принимает целые числа и вычисляет среднее арифметическое четных чисел. Окончание – ввод 0

Листинг кода файла task2.py для выполнения задания 2:

```
import inputValidator

DESCRIPTION = """Create a loop that takes integers and calculates
the average of even numbers. The loop ends when the number 0 is entered"""

def get_numbers(): 1 usage
    """
    Function to get numbers from user

    Returns:
    | List of entered numbers
    """
    list_numbers = []

    while True:
        num = inputValidator.input_data_with_random(description="Input integer num or 0 for finishing input: ", int,
                                                    is_generate_random=True, is_printing_generate_value=True)
        if num != 0:
            list_numbers.append(num)
        else:
            break

    return list_numbers
```

```
def calculate_average_even(numbers): 1 usage
    """
    Function to calculate average of even numbers

    Args:
        numbers: List of integers

    Returns:
        Average value of even numbers or None if no even numbers
    """
    even_numbers = [num for num in numbers if num % 2 == 0]

    if even_numbers:
        return sum(even_numbers) / len(even_numbers)
    else:
        return None

def print_result(average): 1 usage
    """
    Function to display the result

    Args:
        average: Average value to display
    """
    if average is not None:
        print(f"The average of even numbers is: {average}")
    else:
        print("No even numbers were entered")

def print_description(): 1 usage
    """Function for describing the task condition"""
    print(DESCRIPTION, end="\n\n")

def task2(): 1 usage
    """The main function to start the whole process"""
    print_description()
    numbers = get_numbers()
    average = calculate_average_even(numbers)
    print_result(average)
```

Результат выполнения работы:

```
=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 2
Create a loop that takes integers and calculates
the average of even numbers. The loop ends when the number 0 is entered

Input integer num or 0 for finishing input: -
Error: invalid literal for int() with base 10: '-'. Please enter a valid value.
Input integer num or 0 for finishing input: 0.2
Error: invalid literal for int() with base 10: '0.2'. Please enter a valid value.
Input integer num or 0 for finishing input: 3
Input integer num or 0 for finishing input: 6
Input integer num or 0 for finishing input: -2
Input integer num or 0 for finishing input: 1e10
Error: invalid literal for int() with base 10: '1e10'. Please enter a valid value.
Input integer num or 0 for finishing input: 2000
Input integer num or 0 for finishing input: 0
The average of even numbers is: 668.0
```

Задание 3. Не использовать регулярные выражения. В соответствии с заданием своего варианта составить программу для анализа текста, вводимого с клавиатуры.

Вар-т	Условие
9.	В строке, вводимой с клавиатуры, подсчитать количество пробелов и знаков пунктуации

Листинг кода файла task3.py для выполнения задания 3:

```
import inputValidator

DESCRIPTION = """In a line entered from the keyboard,
count the number of spaces and punctuation marks"""

PUNCTUATION_MARKS = '.,;:!?-~"\'()[]{}«»'

def get_input_string(): 1 usage
    """
    Function to get a string from the user

    Returns:
        String entered by user
    """
    return inputValidator.input_data_with_random(description="Input string: ", str,
                                                is_generate_random=True,
                                                is_printing_generate_value=True)

def count_spaces_and_punctuation(string_value): 1 usage
    """
    Function to count spaces and punctuation marks in a string

    Args:
        string_value: Input string

    Returns:
        Tuple of (space_count, punctuation_count)
    """
    space_count = 0
    punctuation_count = 0

    for character in string_value:
        if character == ' ':
            space_count += 1
        elif character in PUNCTUATION_MARKS:
            punctuation_count += 1

    return space_count, punctuation_count

def print_result(space_count, punctuation_count): 1 usage
    """
    Function to display the result

    Args:
        space_count: Number of spaces
        punctuation_count: Number of punctuation marks
    """
    print(f"Number of spaces: {space_count}")
    print(f"Number of punctuation marks: {punctuation_count}")
    print(f"Total: {space_count + punctuation_count}")
```

```
def print_description(): 1 usage
    """Function for describing the task condition"""
    print(DESCRIPTION, end="\n\n")

def task3(): 1 usage
    """The main function to start the whole process"""
    print_description()
    string_value = get_input_string()
    space_count, punctuation_count = count_spaces_and_punctuation(string_value)
    print_result(space_count, punctuation_count)
```

Результат выполнения работы:

```
=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 3
In a line entered from the keyboard,
count the number of spaces and punctuation marks

Input string:
Number of spaces: 0
Number of punctuation marks: 0
Total: 0

=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 3
In a line entered from the keyboard,
count the number of spaces and punctuation marks

Input string: adsdadadaadd, addadasd: -asdsadsadasd %dddd
Number of spaces: 3
Number of punctuation marks: 3
Total: 6
```

Задание 4. Не использовать регулярные выражения. Дана строка текста, в которой слова разделены пробелами и запятыми. В соответствии с заданием своего варианта составьте программу для анализа строки, инициализированной в коде программы:

«So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy and stupid, whether the pleasure of making a daisy-

chain would be worth the trouble of getting up and picking the daisies, when suddenly a White Rabbit with pink eyes ran close by her.»

Если не оговорено иное, то регистр букв при решении задачи не имеет значения.

Вар-т	Условие
9.	а) определить число слов, начинающихся или заканчивающихся на гласную букву; б) определить, сколько раз повторяется каждый символ; в) вывести в алфавитном порядке слова, идущие после запятой

Листинг кода файла task4.py для выполнения задания 4:

```
DESCRIPTION = """a) Determine the number of words starting or ending with a vowel
b) Determine how many times each character is repeated
c) Display words after commas in alphabetical order"""
```

```
CONST_STRING = (
    "So she was considering in her own mind, as well as she could, "
    "for the hot day made her feel very sleepy and stupid, "
    "whether the pleasure of making a daisy-chain "
    "would be worth the trouble of getting up "
    "and picking the daisies, "
    "when suddenly a White Rabbit with pink eyes ran close by her."
)
```

```
VOWELS = "aeiouAEIOU"
```

```
def extract_words(text): 1 usage
    """
```

```
    Extract words from text, removing punctuation
```

```
    Args:
```

```
    |   text: Input text
```

```
    Returns:
```

```
    |   List of words
```

```
    """
```

```
    # Remove punctuation and split into words
```

```
    cleaned_text = text.replace(',', ' ').replace('.', ' ').replace('-', ' ')
```

```
    words = cleaned_text.split()
```

```
    return words
```

```
def count_vowel_words(words): 1 usage
```

```
    """
```

```
    Function to count words starting or ending with a vowel
```

```
    Args:
```

```
    |   words: List of words
```

```
    Returns:
```

```
    |   Count of words starting or ending with vowel
```

```
    """
```

```
    count = 0
```

```
    for word in words:
```

```
        if word: # Check if word is not empty
```

```
            if word[0] in VOWELS or word[-1] in VOWELS:
```

```
                count += 1
```

```
    return count
```

```

def count_character_repeatability(text): 1 usage
    """
    Function to count the frequency of each character in the text

    Args:
        text: Input text

    Returns:
        Dictionary with character frequencies
    """
    char_dict = {}

    for character in text:
        if character in char_dict:
            char_dict[character] += 1
        else:
            char_dict[character] = 1

    return char_dict

def get_words_after_commas(text): 1 usage
    """
    Function to extract and sort words that come after commas

    Args:
        text: Input text

    Returns:
        Sorted list of words after commas
    """
    # Split by commas
    parts = text.split(',')
    words_after_commas = []

    # For each part after comma (skip first part before first comma)
    for i in range(1, len(parts)):
        part = parts[i].strip()
        # Extract first word after comma
        words = part.replace('.', '').split()
        if words:
            words_after_commas.append(words[0].lower())

    # Sort alphabetically
    return sorted(words_after_commas)

def print_description(): 1 usage
    """Function for describing the task condition"""
    print(DESCRIPTION, end="\n\n")

def task4(): 1 usage
    """The main function to start the whole process"""
    print_description()

    print(f"Text: {CONST_STRING}\n")

    # ----- a -----
    words = extract_words(CONST_STRING)
    vowel_word_count = count_vowel_words(words)
    print(f"a) Number of words starting or ending with a vowel: {vowel_word_count}\n")
    # ----- b -----

    char_dict = count_character_repeatability(CONST_STRING)
    print("b) Character repeatability:")
    for char, count in sorted(char_dict.items()):
        print(f"    '{char}': {count}")
    print()

    # ----- c -----
    words_after_commas = get_words_after_commas(CONST_STRING)
    print(f"c) Words after commas in alphabetical order: {words_after_commas}")

```

Результат выполнения работы:

```
=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 4
a) Determine the number of words starting or ending with a vowel
b) Determine how many times each character is repeated
c) Display words after commas in alphabetical order

Text: So she was considering in her own mind, as well as she could, for the hot day made her feel very sleepy

a) Number of words starting or ending with a vowel: 25

b) Character repeatability:
' ': 54
',': 4
'-': 1
'.': 1
'R': 1
'S': 1
'W': 1
'a': 16
'b': 5
'c': 5
'd': 13
'e': 31
'f': 4
'g': 5
'h': 17
'i': 17
'k': 3
'l': 10
'm': 3
'n': 15
'o': 12
'p': 6
'r': 11
's': 15
't': 14
'u': 7
'v': 1
'w': 8
'y': 7

c) Words after commas in alphabetical order: ['as', 'for', 'when', 'whether']
```

Задание 5. В соответствии с заданием своего варианта составить программу для обработки вещественных списков. Программа должна содержать следующие базовые функции:

- 1) ввод элементов списка пользователем;
- 2) проверка корректности вводимых данных;
- 3) реализация основного задания с выводом результатов;
- 4) вывод списка на экран.

Вар-т	Условие
9.	Найти произведение отрицательных элементов списка и сумму положительных элементов списка, расположенных до максимального элемента

Листинг кода файла task5.py для выполнения задания 5:

```
import inputValidator
import randomGenerator

DESCRIPTION = """a) Find the product of negative elements of the list
b) Find the sum of positive elements located before the maximum element"""

def get_numbers(): 1 usage
    """
    Function to get numbers from user

    Returns:
        List of real numbers
    """
    list_numbers = []
    mode = inputValidator.input_data(description="1 - Generate sequence nums\n0 - Manual input\n",
                                     data_type=int, min_value=0, max_value=1)

    if mode == 1:
        count = inputValidator.input_data(description="Input count of elements: ", int, min_value=1, max_value=100)
        list_numbers = list(randomGenerator.generate_sequence(data_type=float, min_value=-100, max_value=100,
                                                             count=count, is_printing_generate_value=False))
        print(f"\nGenerated list: {list_numbers}\n")
    else:
        while True:
            num = inputValidator.input_data_with_random(description="Input float num or 0 for finishing input: ", float,
                                                         is_generate_random=True, is_printing_generate_value=True)
            if num != 0:
                list_numbers.append(num)
            else:
                if not list_numbers:
                    print("List is empty, please enter at least one number")
                    continue
                break

    return list_numbers
```

```

def find_max_index(numbers): 1 usage
    """
    Function to find the index of maximum element

    Args:
        numbers: List of numbers

    Returns:
        Index of maximum element
    """
    max_value = max(numbers)
    return numbers.index(max_value)

def calculate_product_negative(numbers): 1 usage
    """
    Function to calculate the product of negative elements

    Args:
        numbers: List of numbers

    Returns:
        Product of negative elements or None if no negative elements
    """
    product = 1
    has_negative = False

    for num in numbers:
        if num < 0:
            product *= num
            has_negative = True

    return product if has_negative else None

def calculate_sum_positive_before_max(numbers, max_index): 1 usage
    """
    Function to calculate the sum of positive elements before maximum element

    Args:
        numbers: List of numbers
        max_index: Index of maximum element

    Returns:
        Sum of positive elements before max or None if no such elements
    """
    sum_positive = 0
    has_positive = False

    for i in range(max_index):
        if numbers[i] > 0:
            sum_positive += numbers[i]
            has_positive = True

    return sum_positive if has_positive else None

def print_description(): 1 usage
    """Function for describing the task condition"""
    print(DESCRIPTION, end="\n\n")

```

```
def task5(): 1 usage
    """The main function to start the whole process"""
    print_description()

    list_numbers = get_numbers()

    if list_numbers:
        print(f"List: {list_numbers}\n")

        max_index = find_max_index(list_numbers)
        print(f"Maximum element: {list_numbers[max_index]} (index: {max_index})\n")

        # ----- a -----
        product_negative = calculate_product_negative(list_numbers)
        if product_negative is not None:
            print(f"a) Product of negative elements: {product_negative}")
        else:
            print("a) No negative elements in the list")

        # ----- b -----
        sum_positive_before_max = calculate_sum_positive_before_max(list_numbers, max_index)
        if sum_positive_before_max is not None:
            print(f"b) Sum of positive elements before maximum: {sum_positive_before_max}")
        else:
            print("b) No positive elements before maximum element")

    else:
        print("The list is empty")
```

Результат выполнения работы:

```
=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 5
a) Find the product of negative elements of the list
b) Find the sum of positive elements located before the maximum element

1 - Generate sequence nums
0 - Manual input
0
Input float num or 0 for finishing input: asdasdas
Error: could not convert string to float: 'asdasdas'. Please enter a valid value.
Input float num or 0 for finishing input: 1.222
Input float num or 0 for finishing input: 3
Input float num or 0 for finishing input: 0
List: [1.222, 3.0]

Maximum element: 3.0 (index: 1)

a) No negative elements in the list
b) Sum of positive elements before maximum: 1.222
```

```

=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: 5
a) Find the product of negative elements of the list
b) Find the sum of positive elements located before the maximum element

1 - Generate sequence nums
0 - Manual input
1
Input count of elements: h
Error: invalid literal for int() with base 10: 'h'. Please enter a valid value.
Input count of elements: 6

Generated list: [70.4138652327043, 94.28759920501193, -50.79311081428492, 76.92720708395541, 11.18125966358366, 52.894987476612556]

List: [70.4138652327043, 94.28759920501193, -50.79311081428492, 76.92720708395541, 11.18125966358366, 52.894987476612556]

Maximum element: 94.28759920501193 (index: 1)

a) Product of negative elements: -50.79311081428492
b) Sum of positive elements before maximum: 70.4138652327043

```

Листинг кода файла randomGenerator.py для реализации функции генератора:

```

import random
import string

def print_generate_number(generate_value): 2 usages
    """Function for outputting result generate value"""
    print("\nGenerated value:", generate_value, "\n")

def generate_random(data_type, min_value=None, max_value=None, is_printing_generate_value=False): 2 usages
    """
    Function to generate random data based on the specified data type and optional value range constraints

    Args:
        data_type: Type of data to generate (int, float, str)
        min_value: Minimum value for generation
        max_value: Maximum value for generation
        is_printing_generate_value: Flag to print generated value

    Returns:
        Generated random value of specified type
    """
    if data_type == str:
        if min_value is None:
            min_value = 1
        if max_value is None:
            max_value = 10

        generated_value = ''.join(
            random.choices(string.ascii_letters + string.digits + string.punctuation + ' ',
                           k=random.randint(min_value, max_value)))

        if is_printing_generate_value:
            print_generate_number(generated_value)

    return generated_value

```

```

if min_value is None and data_type == float:
    min_value = -1e307
elif min_value is None and data_type == int:
    min_value = -2_147_483_648

if max_value is None and data_type == float:
    max_value = 1e307
elif max_value is None and data_type == int:
    max_value = 2_147_483_647

if data_type == int:
    generated_value = random.randint(min_value, max_value)
elif data_type == float:
    generated_value = random.uniform(min_value, max_value)
else:
    raise ValueError("Unsupported data type. Supported types are int, float, and str.")

if is_printing_generate_value:
    print_generate_number(generated_value)
return generated_value

def generate_sequence(data_type, min_value=None, max_value=None, count=None, is_printing_generate_value=False): 1 usage
    """
    Generator function that yields a sequence of random values

    Args:
        data_type: Type of data to generate
        min_value: Minimum value for generation
        max_value: Maximum value for generation
        count: Number of values to generate
        is_printing_generate_value: Flag to print generated values

    Yields:
        Random values of specified type
    """
    if count is None:
        count = random.randint(a: 10, b: 100)

    for _ in range(count):
        yield generate_random(data_type, min_value, max_value, is_printing_generate_value=is_printing_generate_value)

```

Листинг кода файла inputValidator.py для реализации проверки пользовательских данных:

```

import randomGenerator

def input_data(description, data_type, min_value=None, max_value=None): 3 usages
    """
    Prompts the user for input, validates type and value constraints, and returns the valid input

    Args:
        description: Prompt message for user
        data_type: Expected data type
        min_value: Minimum allowed value
        max_value: Maximum allowed value

    Returns:
        Validated user input
    """
    while True:
        try:
            user_input = data_type(input(description))

            if min_value is not None and user_input < min_value:
                raise ValueError(f"The value have to be greater or = then {min_value}")

```

```

        if max_value is not None and user_input > max_value:
            raise ValueError(f"The value have to be less or = then {max_value}")

        return user_input

    except ValueError as e:
        print(f"Error: {e}. Please enter a valid value.")

def input_data_with_random(description, data_type, min_value=None, max_value=None, 5 usages
                           is_generate_random=False, is_printing_generate_value=False):
    """
    Prompts the user for input, validates type and value constraints, and returns the valid input,
    but generate random value, if user enter word 'random' or 'RANDOM'

    Args:
        description: Prompt message for user
        data_type: Expected data type
        min_value: Minimum allowed value
        max_value: Maximum allowed value
        is_generate_random: Flag to enable random generation
        is_printing_generate_value: Flag to print generated values

    Returns:
        Validated user input or random value
    """
    while True:
        try:
            user_input = input(description)

            if is_generate_random and user_input.upper() == "RANDOM":
                random_value = randomGenerator.generate_random(data_type=data_type, min_value=min_value,
                                                                max_value=max_value,
                                                                is_printing_generate_value=is_printing_generate_value)
                return random_value
            else:
                user_input = data_type(user_input)

        if min_value is not None and user_input < min_value:
            raise ValueError(f"The value have to be greater or = then {min_value}")

        if max_value is not None and user_input > max_value:
            raise ValueError(f"The value have to be less or = then {max_value}")

        return user_input

    except ValueError as e:
        print(f"Error: {e}. Please enter a valid value.")

```

Листинг кода главного файла main.py для модульного запуска программы:

```

import inputValidator
import task1
import task2
import task3
import task4
import task5

def run_task1(): 1 usage
    """Execute Task 1"""
    task1.task1()

def run_task2(): 1 usage
    """Execute Task 2"""
    task2.task2()

```

```

def run_task3(): 1 usage
    """Execute Task 3"""
    task3.task3()

def run_task4(): 1 usage
    """Execute Task 4"""
    task4.task4()

def run_task5(): 1 usage
    """Execute Task 5"""
    task5.task5()

def main(): 1 usage
    """Main function to run the program"""
    while True:
        print("\n" + "=" * 50)
        print("Laboratory Work №3 - Menu")
        print("=" * 50)
        print("1. Task 1: Calculate arccos(x) using series")
        print("2. Task 2: Average of even numbers")
        print("3. Task 3: Count spaces and punctuation")
        print("4. Task 4: Text analysis")
        print("5. Task 5: Process real number list")
        print("0. Exit")
        print("=" * 50)

        select = inputValidator.input_data(description: "Input number of task: ", int, min_value: 0, max_value: 5)

        match select:
            case 1:
                run_task1()
            case 2:
                run_task2()
            case 3:
                run_task3()
            case 4:
                run_task4()
            case 5:
                run_task5()
            case 0:
                print("Exiting program. Goodbye!")
                break

if __name__ == "__main__":
    main()

```

Результат выполнения работы программы:

```

=====
Laboratory Work №3 - Menu
=====
1. Task 1: Calculate arccos(x) using series
2. Task 2: Average of even numbers
3. Task 3: Count spaces and punctuation
4. Task 4: Text analysis
5. Task 5: Process real number list
0. Exit
=====
Input number of task: B
Error: invalid literal for int() with base 10: 'B'. Please enter a valid value.
Input number of task: 1
Write a program to calculate the value of arccos(x)
using its power series expansion:  $\arccos(x) = \pi/2 - \arcsin(x) = \pi/2 - \sum [(2n)! / (4^n * (n!)^2 * (2n+1))] * x^{(2n+1)}$ 
where  $|x| \leq 1$ 

Input value from 0 to 1 for eps:

```